

Simple Command Sequencer Generator

Autocoder User Guide

1 Description

The purpose of this generator is to provide a user-friendly way of defining named command sequences for the Simple Command Sequencer component. The generator takes a YAML model file as input which declares a set of sequences – each a named, ordered list of sub-commands (and optional sleeps) – and autocodes both the Ada specification of the sequence step tables consumed by the sequencer’s Init routine *and* a set of first-class per-sequence wrapper commands so that each sequence appears on the sequencer instance as its own typed, operator-callable command.

Note the example shown in this documentation is used in the unit test of this component so that the reader of this document can see it being used in context. Please refer to the unit test code for more details on how this generator can be used.

2 Schema

The following pykwalify schema is used to validate the input YAML model. Model files must be named in the form *optional_name.assembly_name.command_sequences.yaml*, where *optional_name* is the specific name of this set of sequences (only needed if more than one suite is defined in the assembly) and *assembly_name* is the assembly the sequences will be used in. Generally this file is created in the same directory or near to the assembly model file. The schema is commented to show each available YAML key and what it accomplishes; even without knowing the specifics of pykwalify schemas, you should be able to glean the available structure from the file below.

```
1  # Schema for command_sequences YAML files
2  # This defines the structure and validation rules
3
4  type: map
5  mapping:
6    name:
7      required: true
8      type: str
9      desc: |
10         The name of the command sequences package. This will be used as the
11         Ada package name, so it must follow Ada naming conventions.
12
13    description:
14      required: false
15      type: str
16      desc: |
17         Description of this command sequences package. This will appear in
18         the generated Ada code comments.
19
20    preamble:
21      required: false
22      type: str
23      desc: |
24         Custom preamble text to include in the generated Ada package.
```

```

25
26 command_timeout_seconds:
27     required: false
28     type: int
29     range: { min: 0 }
30     desc: |
31         Default timeout, in whole seconds, that the sequencer will wait for
32         a sub-command's response before declaring the sequence failed. Used
33         for any sequence in this suite that does not specify its own
34         command_timeout_seconds.
35
36 with:
37     required: false
38     type: seq
39     desc: |
40         List of additional Ada packages to "with" in the generated sequences
41         package. Use this when a step's arg expression references a package
42         that is not pulled in automatically, e.g. an enum package for a
43         qualified enumeration literal.
44     sequence:
45         - type: str
46
47 sequences:
48     required: true
49     type: seq
50     desc: |
51         List of command sequence definitions. Each sequence defines a named
52         command that executes a series of sub-commands.
53     sequence:
54         - type: map
55           mapping:
56             name:
57                 required: true
58                 type: str
59                 desc: |
60                     Name of this sequence. This will become a command that can be
61                     executed. Must follow Ada naming conventions.
62
63             description:
64                 required: false
65                 type: str
66                 desc: |
67                     Description of what this sequence does. Will appear in generated
68                     code and documentation.
69
70             arg_type:
71                 required: false
72                 type: str
73                 desc: |
74                     The Ada type for this sequence's argument. If not specified,
75                     the sequence takes no arguments. Must be a valid Ada type name,
76                     possibly qualified (e.g., "Power_Mode.T" or "Packed_U16.T").
77
78             wait_for_command_completion:
79                 required: false
80                 type: bool
81                 default: true
82                 desc: |
83                     Default setting for whether to wait for each sub-command to
84                     complete before executing the next one. Individual steps can
85                     override this.

```

```

86
87     continue_on_failure:
88         required: false
89         type: bool
90         default: false
91         desc: |
92             If true, the sequence will continue executing even if a
93             ↪ sub-command
94             fails. If false, the sequence will abort on the first failure.
95
96     response_behavior:
97         required: false
98         type: str
99         enum: ['send_after_sequence_start',
100             ↪ 'send_after_sequence_completion']
101         desc: |
102             When the sequencer replies to this sequence's own command:
103             send_after_sequence_start (the default) acknowledges as soon as
104             the sequence is started; send_after_sequence_completion defers
105             the reply until the sequence finishes, carrying its final
106             success or failure. This is static per-sequence configuration.
107             Invocations through the generic Run_Sequence command choose
108             their behavior per-call in the command argument instead.
109
110     command_timeout_seconds:
111         required: false
112         type: int
113         range: { min: 0 }
114         desc: |
115             Override the suite-level default timeout for sub-command
116             responses, in whole seconds, for this sequence only.
117
118     sequence:
119         required: true
120         type: seq
121         desc: |
122             The ordered list of steps to execute. Each step is either a
123             command dispatch (`command:` + optional `arg:`) or an in-sequence
124             sleep (`sleep_ms:` + integer milliseconds). The two forms are
125             mutually exclusive; exactly one must be present per step.
126             Validation is enforced in the model loader.
127         sequence:
128             - type: map
129               mapping:
130                 command:
131                     required: false
132                     type: str
133                     desc: |
134                         The command to execute in the format
135                         ↪ "Component.Command_Name".
136                         The component must be a valid component instance name in
137                         ↪ your
138                         assembly, and the command must exist on that component.
139                         Mutually exclusive with `sleep:`.
140
141                 arg:
142                     required: false
143                     type: str
144                     desc: |
145                         An Ada expression that evaluates to the argument for this
146                         command. You can reference the parent sequence's argument

```

```

143         using "Arg.field_name" syntax. If the command takes no
144         argument, omit this field. Only valid alongside
145         ↪ `command:`.
146
147     Examples:
148     - "Arg" - pass through the entire sequence argument
149     - "Arg.Power_Mode" - pass a field from the sequence
150       ↪ argument
151     - "(Mode => Arg.Mode, Enable => True)" - construct a
152       ↪ record
153     - "42" - a literal value
154
155     wait_for_completion:
156     required: false
157     type: bool
158     desc: |
159         Whether to wait for this specific command to complete
160         ↪ before
161         proceeding. If not specified, uses the parent sequence's
162         wait_for_command_completion setting. Only valid alongside
163         `command:`.
164
165     sleep_ms:
166     required: false
167     type: int
168     range: { min: 0, max: 4294967295 }
169     desc: |
170         Number of milliseconds to pause the sequence. Not
171         bus-routed; the sequencer frame parks itself for the
172         duration without issuing any command. Mutually exclusive
173         with `command:`/`arg:`/`wait_for_completion:`.
174
175     Example:
176     - sleep_ms: 3000 # sleep 3 seconds
177
178     # Additional validation rules (enforced in the model Python code):
179     # - Command format must be "Component.Command" with valid identifiers, or
180     #   the reserved name "Sleep"
181     # - Referenced types in arg_type must exist in the system
182     # - Argument expressions must be valid Ada syntax
183     # - Component and command names must match instances in the assembly

```

3 Example Input

The following is an example command sequences input YAML file. It defines five sequences exercising the main features: an arg-less sequence with both statically-armed commands and a sleep, a sequence with a typed argument record whose fields are dispatched as dynamic arguments to two sub-commands, a no-wait sequence used for direct-dispatch testing, a sequence with a single large sleep that exercises the out-of-range protection, and a sequence that calls another sequence as a sub-sequence step. This example adheres to the schema shown in the previous section, and is commented to give clarification.

Note the optional top-level `with:` key, which lists additional Ada packages to “with” in the generated package. A step’s `arg:` expression is emitted verbatim into the generated Ada, so any package it references that is not pulled in automatically – such as an enum package needed to qualify an enumeration literal – must be listed here.

```

1  # Command Sequences Definition
2  # This file defines composite commands that execute sequences of sub-commands
3
4  name: Flight_Sequences
5  description: Common flight software command sequences
6
7  command_timeout_seconds: 10  # default wait for sub-command responses
8
9  sequences:
10     # Simple sequence with no arguments
11     - name: Sequence_A
12       description: Execute commands on component A
13       wait_for_command_completion: true
14       continue_on_failure: true
15
16       sequence:
17         - command: Component_A.Command_1
18         - command: Component_A.Command_2
19         arg: (Seconds => 3, Subseconds => 14)
20         - sleep_ms: 3000
21         - command: Component_A.Command_3
22         arg: (Value => 3)
23
24     - name: Sequence_B
25       description: Perform generic arguments with component A and B
26       wait_for_command_completion: true
27       continue_on_failure: false
28       arg_type: Sequence_B_Arg.T
29
30       sequence:
31         - command: Component_A.Command_3
32         arg: Arg.Component_A_Arg
33         - command: Component_B.Command_3
34         arg: Arg.Component_B_Arg
35
36     - name: Sequence_C
37       description: No-wait sequence for direct-dispatch testing
38       wait_for_command_completion: false
39       continue_on_failure: false
40
41       sequence:
42         - command: Component_A.Command_1
43         - command: Component_A.Command_1
44
45     - name: Sequence_D
46       description: Sequence with a large sleep value to test an overflow sleep
47       wait_for_command_completion: true
48       continue_on_failure: false
49
50       sequence:
51         - sleep_ms: 4294967295
52
53     # A sequence that calls another sequence. Each sequence is a first-class
54     # command on the sequencer, so a sub-sequence step is just a command step
55     # targeting the sequencer's own synthesized command. The sub-sequence's
56     # response behavior is its own static configuration (Sequence_C defaults to
57     # send_after_sequence_start, so its reply arrives as soon as it starts).
58     - name: Sequence_E
59       description: Call Sequence_C as a sub-sequence, then run a regular command
60       wait_for_command_completion: true
61       continue_on_failure: false

```

```

62
63     sequence:
64         - command: Sequencer.Sequence_C
65         - command: Component_A.Command_1
66
67     # A sequence whose command response is statically deferred until the
68     # sequence completes -- the operator's reply carries the final
69     # success/failure instead of a start acknowledgement.
70     - name: Sequence_F
71       description: No-wait sequence with a statically deferred command response
72       wait_for_command_completion: false
73       continue_on_failure: false
74       response_behavior: send_after_sequence_completion
75
76     sequence:
77         - command: Component_A.Command_1

```

4 Example Output

The example input above produces the following Ada outputs. The package spec declares the per-dynamic-step Resolver types, a step array constant per sequence, and the Sequences_Table constant (plus a stable Sequences access pointer). The package body implements the Resolver Resolve functions, which deserialize the sequence's per-call argument buffer through the input type's `Serialization.From_Byte_Array` and re-serialize the addressed field as the sub-command's argument buffer. The body is only emitted when the suite contains at least one dynamic step – a suite of purely static or sleep-only sequences declares no Resolve subprograms and so needs no body package.

The Sequences access should be passed into the Simple Command Sequencer's Init procedure during assembly initialization, e.g. in the assembly YAML:

```

- type: Simple_Command_Sequencer
  name: My_Sequencer
  init:
    - "Num_Concurrent_Sequences => 4"
    - "Sequences => Test_Assembly_Command_Sequences_Example_Sequences.Sequences"

```

```

1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  -- Common flight software command sequences
4  with Simple_Sequencer_Types; use Simple_Sequencer_Types;
5  with Test_Assembly_Commands; use Test_Assembly_Commands;
6  with Command;
7  with Command_Types;
8  with Basic_Types;
9  with Sequence_Enums;
10 with Sequence_Arg_Utills; use Sequence_Arg_Utills;
11 with Sys_Time_32;
12 with Packed_U32;
13 with Sequence_B_Arg;
14 package Test_Assembly_Command_Sequences_Example_Sequences is
15     -----
16     -- Resolver types - one per dynamic step, each encodes a traversal path.
17     -----
18     function Sequence_B_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
19     ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
19     function Sequence_B_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
20     ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
20     -----

```

```

21  -- Step arrays - one constant per sequence.
22  -----
23  Sequence_A_Steps : aliased constant Step_Array :=
24  [
25      0 => (Kind          => Command_Step,
26             Id           => Component_A_Command_1,
27             Arg_Length => 0,
28             Arg          => [others => 0]),
29      1 => (Kind          => Command_Step,
30             Id           => Component_A_Command_2,
31             Arg_Length =>
32             ↪ Sys_Time_32.Serialization.Serialized_Length,
33             Arg          => To_Arg
34             ↪ (Sys_Time_32.Serialization.To_Byte_Array
35             ↪ ((Seconds => 3, Subseconds => 14)))),
36      2 => (Kind          => Sleep,
37             Id           => 0,
38             Arg_Length => 0,
39             Sleep_Arg    => (Value => 3000)),
40      3 => (Kind          => Command_Step,
41             Id           => Component_A_Command_3,
42             Arg_Length =>
43             ↪ Packed_U32.Serialization.Serialized_Length,
44             Arg          => To_Arg
45             ↪ (Packed_U32.Serialization.To_Byte_Array ((Value
46             ↪ => 3))))
47  ];
48  Sequence_B_Steps : aliased constant Step_Array :=
49  [
50      0 => (Kind          => Runtime_Argument_Command_Step,
51             Id           => Component_A_Command_3,
52             Arg_Length =>
53             ↪ Packed_U32.Serialization.Serialized_Length,
54             Resolver    => Sequence_B_Step_0_Resolver_T'Access),
55      1 => (Kind          => Runtime_Argument_Command_Step,
56             Id           => Component_B_Command_3,
57             Arg_Length =>
58             ↪ Packed_U32.Serialization.Serialized_Length,
59             Resolver    => Sequence_B_Step_1_Resolver_T'Access)
60  ];
61  Sequence_C_Steps : aliased constant Step_Array :=
62  [
63      0 => (Kind          => Command_Step,
64             Id           => Component_A_Command_1,
65             Arg_Length => 0,
66             Arg          => [others => 0]),
67      1 => (Kind          => Command_Step,
68             Id           => Component_A_Command_1,
69             Arg_Length => 0,
70             Arg          => [others => 0])
71  ];
72  Sequence_D_Steps : aliased constant Step_Array :=
73  [
74      0 => (Kind          => Sleep,
75             Id           => 0,
76             Arg_Length => 0,
77             Sleep_Arg    => (Value => 4294967295))
78  ];
79  Sequence_E_Steps : aliased constant Step_Array :=
80  [
81      0 => (Kind          => Command_Step,

```

```

74         Id          => Sequencer_Sequence_C,
75         Arg_Length => 0,
76         Arg          => [others => 0]),
77     1 => (Kind          => Command_Step,
78         Id          => Component_A_Command_1,
79         Arg_Length => 0,
80         Arg          => [others => 0])
81 ];
82 Sequence_F_Steps : aliased constant Step_Array :=
83 [
84     0 => (Kind          => Command_Step,
85         Id          => Component_A_Command_1,
86         Arg_Length => 0,
87         Arg          => [others => 0])
88 ];
89 -----
90 -- Sequences_Table
91 -----
92 Sequences_Table : aliased constant Sequences_Type :=
93 [
94     0 => (
95         Wait_For_Cmd_Resp      => True,
96         Abort_On_Failed_Cmd    => False,
97         Command_Timeout_Millis => 10000,
98         Response_Behavior      => Sequence_Enums.Sequence_Response_Behavior.Se_
99         ↪ nd_After_Sequence_Start,
100        Steps                  => Sequence_A_Steps'Access),
101     1 => (
102         Wait_For_Cmd_Resp      => True,
103         Abort_On_Failed_Cmd    => True,
104         Command_Timeout_Millis => 10000,
105         Response_Behavior      => Sequence_Enums.Sequence_Response_Behavior.Se_
106         ↪ nd_After_Sequence_Start,
107        Steps                  => Sequence_B_Steps'Access),
108     2 => (
109         Wait_For_Cmd_Resp      => False,
110         Abort_On_Failed_Cmd    => True,
111         Command_Timeout_Millis => 10000,
112         Response_Behavior      => Sequence_Enums.Sequence_Response_Behavior.Se_
113         ↪ nd_After_Sequence_Start,
114        Steps                  => Sequence_C_Steps'Access),
115     3 => (
116         Wait_For_Cmd_Resp      => True,
117         Abort_On_Failed_Cmd    => True,
118         Command_Timeout_Millis => 10000,
119         Response_Behavior      => Sequence_Enums.Sequence_Response_Behavior.Se_
120         ↪ nd_After_Sequence_Start,
121        Steps                  => Sequence_D_Steps'Access),
122     4 => (
123         Wait_For_Cmd_Resp      => True,
124         Abort_On_Failed_Cmd    => True,
125         Command_Timeout_Millis => 10000,
126         Response_Behavior      => Sequence_Enums.Sequence_Response_Behavior.Se_
127         ↪ nd_After_Sequence_Start,
128        Steps                  => Sequence_E_Steps'Access),
129     5 => (
130         Wait_For_Cmd_Resp      => False,
131         Abort_On_Failed_Cmd    => True,
132         Command_Timeout_Millis => 10000,
133         Response_Behavior      => Sequence_Enums.Sequence_Response_Behavior.Se_
134         ↪ nd_After_Sequence_Completion,

```



```

129     Steps                      => Sequence_F_Steps'Access)
130 ];
131
132 Sequences : constant Sequences_Access := Sequences_Table'Access;
133
134 -----
135 -- Command builders for the per-sequence ("ghost") commands. These commands
136 -- are first-class in the assembly command dictionary but have no generated
137 -- Ada handler on the component, so this surface reconstructs the
138 -- operator-side builders (id getters + Command.T constructors) for unit
139 -- tests and other on-board callers.
140 --
141 -- Set Id_Base to the sequencer instance's command id base (the value passed
142 -- to Set_Id_Bases). Each sequence command id is then
143 -- Id_Base + Simple_Command_Sequencer_Commands.Num_Commands + declaration
144 -- index.
145 -----
146 type Instance is tagged private;
147
148 -- Set the sequencer instance's command id base so constructed commands
149 -- ↪ carry
150 -- ids that match what the component registered at runtime.
151 procedure Set_Id_Base (Self : in out Instance; Id_Base : in
152     ↪ Command_Types.Command_Id);
153
154 -- Set the operator source id stamped into the header of constructed commands
155 -- (defaults to 0). Mirrors the command suite's Set_Source_Id.
156 procedure Set_Source_Id (Self : in out Instance; Source_Id : in
157     ↪ Command_Types.Command_Source_Id);
158
159 function Get_Sequence_A_Id (Self : in Instance) return
160     ↪ Command_Types.Command_Id;
161 function Sequence_A (Self : in Instance) return Command.T;
162 function Get_Sequence_B_Id (Self : in Instance) return
163     ↪ Command_Types.Command_Id;
164 function Sequence_B (Self : in Instance; Arg : in Sequence_B_Arg.T) return
165     ↪ Command.T;
166 function Get_Sequence_C_Id (Self : in Instance) return
167     ↪ Command_Types.Command_Id;
168 function Sequence_C (Self : in Instance) return Command.T;
169 function Get_Sequence_D_Id (Self : in Instance) return
170     ↪ Command_Types.Command_Id;
171 function Sequence_D (Self : in Instance) return Command.T;
172 function Get_Sequence_E_Id (Self : in Instance) return
173     ↪ Command_Types.Command_Id;
174 function Sequence_E (Self : in Instance) return Command.T;
175 function Get_Sequence_F_Id (Self : in Instance) return
176     ↪ Command_Types.Command_Id;
177 function Sequence_F (Self : in Instance) return Command.T;
178
179 private
180     type Instance is tagged record
181         Id_Base : Command_Types.Command_Id := 0;
182         Source_Id : Command_Types.Command_Source_Id := 0;
183     end record;
184 end Test_Assembly_Command_Sequences_Example_Sequences;

```

```

1 -- Auto-generated by Adamant command sequences generator.
2 -- Do not edit by hand.
3 with Simple_Command_Sequencer_Commands;

```

```

4  with Interfaces;
5  with Sequence_B_Arg.Validation;
6  package body Test_Assembly_Command_Sequences_Example_Sequences is
7
8      -- Make the "+" operator on Command_Id visible for the id arithmetic below.
9      use type Command_Types.Command_Id;
10
11      -----
12      -- Resolvers - one per dynamic step. Each validates the sequence's per-call
13      -- argument buffer, deserializes it as the sequence's arg type, and returns
14      -- the serialized leaf field for its sub-command's Arg_Buffer.
15      -----
16      function Sequence_B_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
17      ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
18          Errant_Field : Interfaces.Unsigned_32;
19          Input : Sequence_B_Arg.T;
20          Valid : Boolean := Sequence_B_Arg.Validation.Valid (Bytes (Bytes'First ..
21          ↪ Bytes'First + Sequence_B_Arg.Serialization.Serialized_Length - 1),
22          ↪ Errant_Field);
23      begin
24          if Valid then
25              Input := Sequence_B_Arg.Serialization.From_Byte_Array (
26                  Bytes (Bytes'First .. Bytes'First +
27                  ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1));
28              Args := To_Arg (
29                  Packed_U32.Serialization.To_Byte_Array (
30                  ↪ Input.Component_A_Arg
31                  )
32              );
33          end if;
34          return Valid;
35      end Sequence_B_Step_0_Resolver_T;
36      function Sequence_B_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
37      ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
38          Errant_Field : Interfaces.Unsigned_32;
39          Input : Sequence_B_Arg.T;
40          Valid : Boolean := Sequence_B_Arg.Validation.Valid (Bytes (Bytes'First ..
41          ↪ Bytes'First + Sequence_B_Arg.Serialization.Serialized_Length - 1),
42          ↪ Errant_Field);
43      begin
44          if Valid then
45              Input := Sequence_B_Arg.Serialization.From_Byte_Array (
46                  Bytes (Bytes'First .. Bytes'First +
47                  ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1));
48              Args := To_Arg (
49                  Packed_U32.Serialization.To_Byte_Array (
50                  ↪ Input.Component_B_Arg
51                  )
52              );
53          end if;
54          return Valid;
55      end Sequence_B_Step_1_Resolver_T;
56
57      -----
58      -- Command builders
59      -----
60      procedure Set_Id_Base (Self : in out Instance; Id_Base : in
61      ↪ Command_Types.Command_Id) is
62      begin
63          Self.Id_Base := Id_Base;
64      end Set_Id_Base;
65
66

```

```

56 procedure Set_Source_Id (Self : in out Instance; Source_Id : in
   ↪ Command_Types.Command_Source_Id) is
57 begin
58     Self.Source_Id := Source_Id;
59 end Set_Source_Id;
60
61 function Get_Sequence_A_Id (Self : in Instance) return
   ↪ Command_Types.Command_Id is
62 begin
63     return Self.Id_Base
64         + Command_Types.Command_Id
65         ↪ (Simple_Command_Sequencer_Commands.Num_Commands)
66         + Command_Types.Command_Id (0);
67 end Get_Sequence_A_Id;
68
69 function Sequence_A (Self : in Instance) return Command.T is
70 begin
71     return (Header => (Source_Id => Self.Source_Id,
72                       Id => Self.Get_Sequence_A_Id,
73                       Arg_Buffer_Length => 0),
74            Arg_Buffer => [others => 0]);
75 end Sequence_A;
76
77 function Get_Sequence_B_Id (Self : in Instance) return
   ↪ Command_Types.Command_Id is
78 begin
79     return Self.Id_Base
80         + Command_Types.Command_Id
81         ↪ (Simple_Command_Sequencer_Commands.Num_Commands)
82         + Command_Types.Command_Id (1);
83 end Get_Sequence_B_Id;
84
85 function Sequence_B (Self : in Instance; Arg : in Sequence_B_Arg.T) return
   ↪ Command.T is
86 begin
87     Bytes : constant Basic_Types.Byte_Array :=
88         ↪ Sequence_B_Arg.Serialization.To_Byte_Array (Arg);
89     Buffer : Command_Types.Command_Arg_Buffer_Type := [others => 0];
90     Buffer (Buffer'First .. Buffer'First + Bytes'Length - 1) := Bytes;
91     return (Header => (Source_Id => Self.Source_Id,
92                       Id => Self.Get_Sequence_B_Id,
93                       Arg_Buffer_Length =>
94                           ↪ Command_Types.Command_Arg_Buffer_Length_Type
95                           ↪ (Bytes'Length)),
96            Arg_Buffer => Buffer);
97 end Sequence_B;
98
99 function Get_Sequence_C_Id (Self : in Instance) return
   ↪ Command_Types.Command_Id is
100 begin
101     return Self.Id_Base
102         + Command_Types.Command_Id
103         ↪ (Simple_Command_Sequencer_Commands.Num_Commands)
104         + Command_Types.Command_Id (2);
105 end Get_Sequence_C_Id;
106
107 function Sequence_C (Self : in Instance) return Command.T is
108 begin
109     return (Header => (Source_Id => Self.Source_Id,
110                       Id => Self.Get_Sequence_C_Id,
111                       Arg_Buffer_Length => 0),
112            Arg_Buffer => Buffer);
113 end Sequence_C;

```

```

106         Arg_Buffer => [others => 0]);
107     end Sequence_C;
108
109     function Get_Sequence_D_Id (Self : in Instance) return
110     ↪ Command_Types.Command_Id is
111     begin
112         return Self.Id_Base
113             + Command_Types.Command_Id
114             ↪ (Simple_Command_Sequencer_Commands.Num_Commands)
115             + Command_Types.Command_Id (3);
116     end Get_Sequence_D_Id;
117
118     function Sequence_D (Self : in Instance) return Command.T is
119     begin
120         return (Header => (Source_Id => Self.Source_Id,
121             Id => Self.Get_Sequence_D_Id,
122             Arg_Buffer_Length => 0),
123             Arg_Buffer => [others => 0]);
124     end Sequence_D;
125
126     function Get_Sequence_E_Id (Self : in Instance) return
127     ↪ Command_Types.Command_Id is
128     begin
129         return Self.Id_Base
130             + Command_Types.Command_Id
131             ↪ (Simple_Command_Sequencer_Commands.Num_Commands)
132             + Command_Types.Command_Id (4);
133     end Get_Sequence_E_Id;
134
135     function Sequence_E (Self : in Instance) return Command.T is
136     begin
137         return (Header => (Source_Id => Self.Source_Id,
138             Id => Self.Get_Sequence_E_Id,
139             Arg_Buffer_Length => 0),
140             Arg_Buffer => [others => 0]);
141     end Sequence_E;
142
143     function Get_Sequence_F_Id (Self : in Instance) return
144     ↪ Command_Types.Command_Id is
145     begin
146         return Self.Id_Base
147             + Command_Types.Command_Id
148             ↪ (Simple_Command_Sequencer_Commands.Num_Commands)
149             + Command_Types.Command_Id (5);
150     end Get_Sequence_F_Id;
151
152     function Sequence_F (Self : in Instance) return Command.T is
153     begin
154         return (Header => (Source_Id => Self.Source_Id,
155             Id => Self.Get_Sequence_F_Id,
156             Arg_Buffer_Length => 0),
157             Arg_Buffer => [others => 0]);
158     end Sequence_F;
159
160 end Test_Assembly_Command_Sequences_Example_Sequences;

```

5 Per-Sequence First-Class Commands

In addition to the sequence tables above, this same model drives the autocoded synthesis of one first-class command per declared sequence on the Simple Command Sequencer instance. Discovery is build-path-driven: the sequencer's commands model queries the model database for every `*.command_sequences.yaml` in scope, so a naked component build (no assembly) sees no sequences and produces only the four static commands (`Run_Sequence`, `Kill_All_Sequences`, `Kill_Frame`, `Set_Summary_Packet_Period`), while an assembly build whose path includes a sequences suite adds one command per sequence. Each per-sequence command (e.g. `Sequence_B`) takes the sequence's own `arg_type` verbatim as its command argument (or no argument for an argless sequence) – a user-written, normally-registered type, so no argument records are generated or registered anywhere in the pipeline. At run time the component copies the received argument buffer unchanged into a generic `Run_Sequence_Arg.T` and dispatches through the existing `Run_Sequence` backbone using the sequence's runtime slot index; the response behavior comes from the sequence's static configuration in the autocoded table (see Sequence Response Behavior below). The generated package also carries a builder surface (per-sequence command id getters and `Command.T` constructors) for unit tests and other on-board callers – no per-sequence Ada needs to be hand-written.

6 Sequences Calling Sequences

Because every sequence is a first-class command on the sequencer instance, a sequence step may invoke another sequence simply by targeting the sequencer's own synthesized command, e.g. `command: My_Sequencer.Sequence_C`. No special syntax or generator support is involved – the step resolves like any other `Component.Command` reference, provided the sequencer instance is declared in the assembly. At run time the sub-sequence command is emitted out the sequencer's `Command_T_Send` connector, routed through the command router, and arrives back at the sequencer's `Command_T_Recv_Async`, where it claims its own frame and executes concurrently with (and independently of) the calling sequence. If the calling sequence waits for command completion, the sub-sequence's command response (per its static `response_behavior` configuration, see below) is what wakes it. Nesting depth is naturally capped by `Num_Concurrent_Sequences`: a sub-sequence call when no frame is available is rejected with a `No_Frame_Available` event and a `Failure` response to the calling frame.

7 Sequence Response Behavior

When the sequencer replies to the command that started a sequence is controlled by the `Sequence_Response_Behavior` enumeration. For the synthesized per-sequence commands it is static configuration: the optional `response_behavior` field on each sequence in the YAML, baked into the autocoded sequence table. The generic `Run_Sequence` command instead carries the behavior as an argument, so a per-call override is always available through it:

- `Send_After_Sequence_Start` (the conventional default) – the command response is sent immediately once the sequence has been accepted and started on a frame. This is the right choice for fire-and-forget operation and for sub-sequence calls where the caller should proceed as soon as the sub-sequence is running.
- `Send_After_Sequence_Completion` – the command response is withheld until the sequence ends, and its status reflects the outcome: `Success` when the final step completes, or `Failure` when the sequence is aborted by a failed sub-command, times out waiting for a sub-command response, encounters an out-of-range sleep or timeout, or is halted by `Kill_All_Sequences`. The sequencer captures the originating command's source and id per frame, so concurrent deferred sequences each reply to their own operator.

A sequence step may forward a field of the parent sequence's argument to the sub-command it dispatches, using the `Arg.field.subfield` syntax in the step's `arg:`. Each such reference produces a typed Resolver in the generated package: at run time the sequencer dispatches that Resolver, which

deserializes the per-call argument buffer through the input type's safe `Serialization.From_Byte_Array`, navigates to the addressed field, and serializes it as the sub-command's argument buffer. The resolver's signature accepts only a `Basic_Types.Byte_Array` – there is no 'Address reinterpretation' – so the dynamic-argument machinery is type-safe by construction; a resolver cannot overlay an arbitrary record onto raw memory, only deserialize a byte array through the input type's generated deserializer.

8 Sleep Steps

A sequence step may be a sleep instead of a command: `sleep_ms: 3000` pauses the executing frame for 3 seconds without issuing any bus traffic. Sleep is not a bus command; it is interpreted intrinsically by the sequencer frame state machine. The duration is a plain integer number of milliseconds (up to $2^{32} - 1$), fixed at build time. Out-of-range sleep durations (those that cannot be represented as a `Time_Span` on the platform, or that would overflow `Sys_Time` arithmetic when added to the current time) are rejected at run time with a `Sequence_Out_Of_Range_Sleep` event so the operator can see that the step was skipped.